

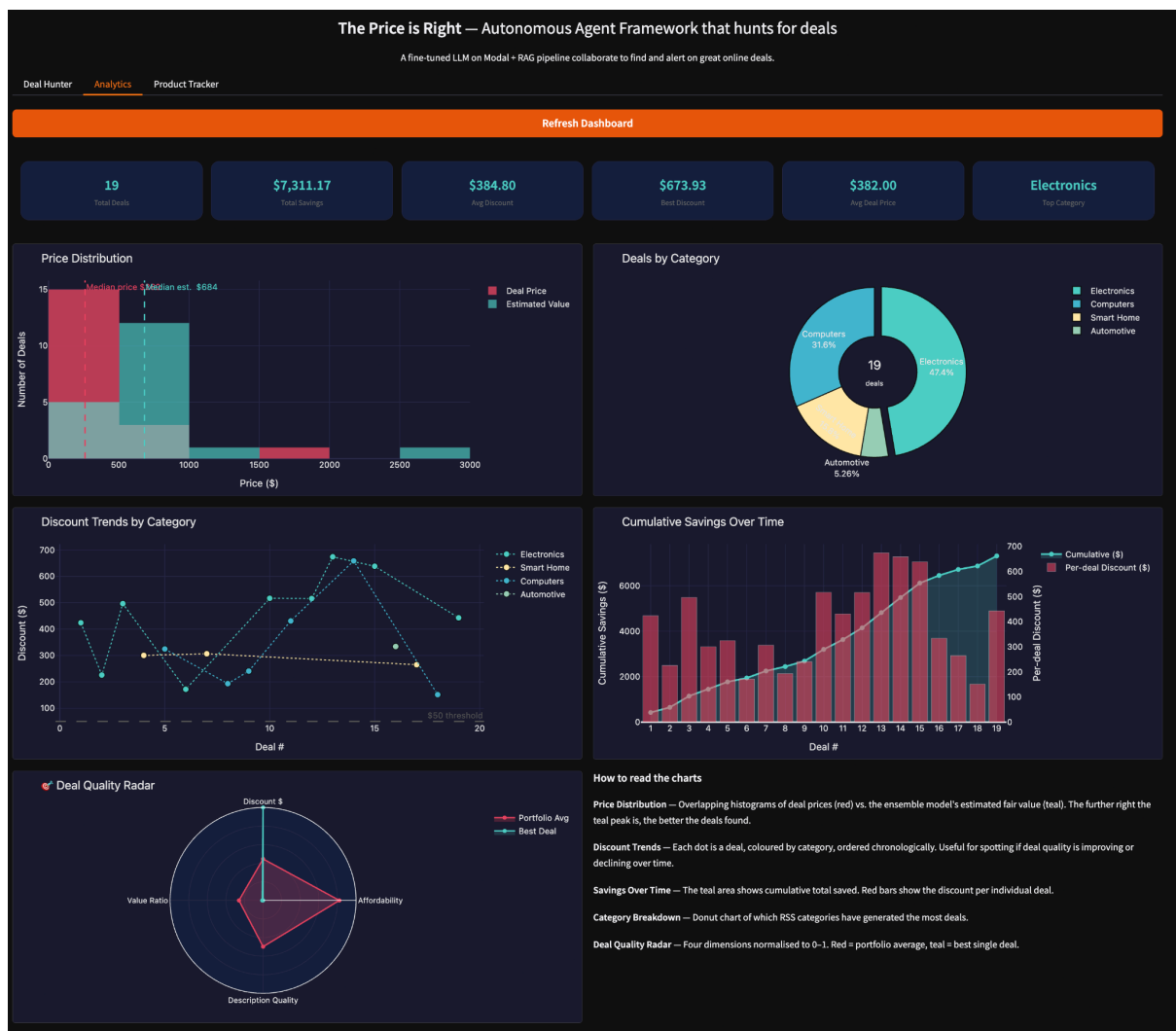
Ingeniería de LLMs - Proyecto Final

Se añaden dos nuevas funcionalidades al proyecto que se describen a continuación y algunas herramientas que ayudan al despliegue de entornos y la ejecución de la aplicación.

1. Dashboard

Se añade una capa de análisis a una pestaña que permite visualizar indicadores a través de gráficos.

Esta capa de análisis lee la misma lista de oportunidades (List[Opportunity]) en memoria que gestiona el `deal_agent_framework.py`. Los gráficos se generan cuando el usuario hace click en “Refresh Dashboard” o cuando Gradio carga la pestaña por primera vez.



Para generar las gráficas se ha usado la librería Plotly que ya era una dependencia del proyecto. Plotly proporciona de forma nativa algunas funcionalidades interesantes como: zoom, descarga del gráfico como png, seleccionar un área o autoescalar.

Se crea el fichero [charts.py](#) que incluye todas las funciones para generar los gráficos.

- `price_distribution()`: Histograma superpuesto (gráfico 1).
- `discount_trends()`: Dispersión más líneas por categoría (gráfico 2).
- `deal_quality_radar()`: Gráfico de radar polar (gráfico 3).
- `savings_over_time()`: Área más barras de doble eje (gráfico 4).
- `category_breakdown()`: Gráfico de rosca (gráfico 5).
- `summary_stats_html()`: Tarjetas KPI como cadena HTML.

La pestaña “Analytics” se añade al fichero `price_is_right_final.py` y dentro la interfaz de Gradio:

```
with gr.Tab("Analytics"):
    with gr.Row():
        refresh_analytics_btn = gr.Button(
            "Refresh Dashboard", variant="primary", scale=0
        )

    # KPI summary cards
    analytics_kpi = gr.HTML()

    # Row 1: Price Distribution | Category Breakdown
    with gr.Row():
        with gr.Column(scale=1):
            chart_price_dist = gr.Plot(show_label=False)
        with gr.Column(scale=1):
            chart_category = gr.Plot(show_label=False)

    # Row 2: Discount Trends | Savings Over Time
    with gr.Row():
        with gr.Column(scale=1):
            chart_discount_trends = gr.Plot(show_label=False)
        with gr.Column(scale=1):
            chart_savings = gr.Plot(show_label=False)

    # Row 3: Radar (full width, centred)
    with gr.Row():
        with gr.Column(scale=1):
            chart_radar = gr.Plot(show_label=False)
        with gr.Column(scale=1):
            gr.Markdown(
                """
                ### How to read the charts\n\n
                **Price Distribution** – Overlapping histograms of deal prices "
                (red) vs. the ensemble model's estimated fair value (teal). "
                The further right the teal peak is, the better the deals
            """
            )
```

```

found.\n\n"

    """Discount Trends** – Each dot is a deal, coloured by category, "
    "ordered chronologically. Useful for spotting if deal quality is "
    "improving or declining over time.\n\n"

    """Savings Over Time** – The teal area shows cumulative total
saved. "

    "Red bars show the discount per individual deal.\n\n"
    """Category Breakdown** – Donut chart of which RSS categories "
    "have generated the most deals.\n\n"
    """Deal Quality Radar** – Four dimensions normalised to 0-1. "
    "Red = portfolio average, teal = best single deal."

)

def load_analytics():
    mem = self.get_agent_framework().memory
    return (
        ch.summary_stats_html(mem),
        ch.price_distribution(mem),
        ch.category_breakdown(mem),
        ch.discount_trends(mem),
        ch.savings_over_time(mem),
        ch.deal_quality_radar(mem),
    )

analytics_outputs = [
    analytics_kpi,
    chart_price_dist,
    chart_category,
    chart_discount_trends,
    chart_savings,
    chart_radar,
]

ui.load(load_analytics, outputs=analytics_outputs)
refresh_analytics_btn.click(load_analytics, outputs=analytics_outputs)

```

Los gráficos añadidos se describen a continuación:

1. Distribución de precios:

- **Tipo:** Histograma superpuesto (barmode="overlay")
- **Datos:** deal.price (rojo) vs estimate (verde azulado) para todas las ofertas en memoria
- **Información:** Cuánto por debajo del precio justo estimado por el modelo están las ofertas. Si el pico de color verde azulado (estimación) se sitúa a la

derecha del pico rojo (precio), el agente encuentra ofertas consistentemente por debajo del mercado.

2. Tendencias de descuento:

- **Tipo:** Dispersión + líneas conectoras discontinuas
- **Datos:** discount por oferta, coloreado por categoría de feed RSS, ordenado por índice de oferta.
- **Información:** Si la calidad de la oferta (tamaño del descuento) mejora o se deteriora con el tiempo, y qué categorías tienen el mejor rendimiento.

3. Radar de calidad de oferta

- **Tipo:** go.Scatterpolar con fill="toself"
- **Datos:** Cuatro dimensiones de calidad normalizadas, dos trazas
- **Información:** Comparación de calidad multidimensional entre el promedio de la cartera y la mejor oferta encontrada.

4. Ahorros a lo largo del tiempo

- **Tipo:** Gráfico de doble eje (plotly.subplots.make_subplots(secondary_y=True))
- **Datos:** Ahorros acumulados (eje Y izquierdo, área verde azulada) + descuento por oferta (eje Y derecho, barras rojas)
- **Información:** Tanto la tasa de acumulación de ahorros como la distribución de los valores individuales de las ofertas.

5. Desglose por categoría

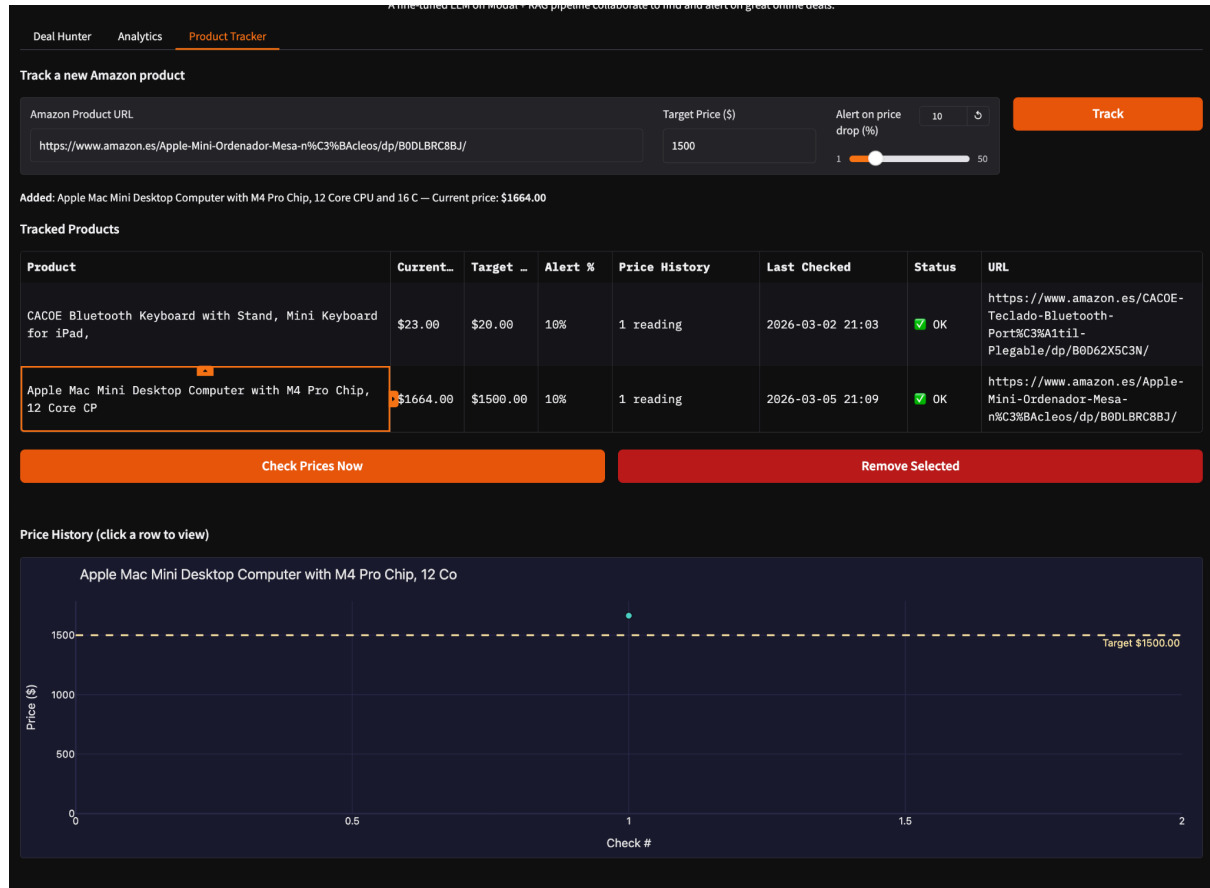
- **Tipo:** Gráfico de rosca (go.Pie con hole=0.42)
- **Datos:** Recuento de ofertas por categoría RSS
- **Información:** En qué categorías de productos se centra más el escáner.

6. Tarjetas KPI de resumen

- summary_stats_html() devuelve una cadena HTML de flex-box con seis tarjetas de métricas:
 - Total de ofertas: len(memory)
 - Ahorros totales: sum(max(discount, 0))
 - Descuento promedio: total_savings / len(memory)
 - Mejor descuento: max(o.discount for o in memory)
 - Precio promedio de oferta: mean(o.deal.price)
 - Categoría principal: Moda de las categorías inferidas.

2. Product Tracker

Se añade una pestaña nueva que permite a los usuarios añadir manualmente URLs de productos de Amazon y recibir alertas cuando el precio baja o alcanza un valor objetivo.



Mientras que el cazador de ofertas descubre ofertas aleatorias de feeds RSS, el rastreador permite al usuario elegir productos específicos y seguir sus precios a lo largo del tiempo.

Flujo de trabajo:

1. El usuario introduce una URL de Amazon.
2. ProductTrackerAgent extrae datos de la página (título, precio, disponibilidad, etc.).
3. TrackedProduct guardado en tracked_products.json
4. Cada vez que se ejecuta "Check Prices Now" se lanzan alertas si:
 - a. El precio es menor o igual al precio objetivo
 - b. El precio bajó $\geq$ % objetivo desde el inicio
 - c. Palabras clave de bajo stock

La nueva funcionalidad sigue el mismo patrón de agentes ya utilizado en el proyecto. Una subclase Agent dedicada gestiona toda la E/S externa (scraping HTTP), mientras que DealAgentFramework gestiona el ciclo de vida de los datos (persistencia, CRUD, programación).

El nuevo agente se encuentra en el fichero `product_tracker_agent.py`. La clase `ProductTrackerAgent` tiene los siguientes métodos:

- `scrape_amazon`
- `_extract_title`
- `_extract_price`
- `_extract_availability`
- `_extract_image`
- `_parse_price`
- `_check_alerts`
- `check_product`
- `check_all`

Se añade un nuevo modelo Pydantic `TrackedProduct` al fichero `agents/deals.py`:

```
class TrackedProduct(BaseModel):  
    """  
    A product that the user explicitly wants to monitor.  
    Stores the current price, a configurable target price, an alert threshold  
    (% drop from initial price), and the full price history.  
    """  
    url: str  
    title: str = "Pending..."  
    image_url: Optional[str] = None  
    current_price: Optional[float] = None  
    target_price: Optional[float] = None  
    alert_threshold_pct: float = 10.0 # Alert when price drops this % from  
    initial  
    price_history: List[Dict] = [] # [{"timestamp": ISO str, "price": float}]  
    last_checked: Optional[str] = None  
    added_at: str = ""  
    scrape_error: Optional[str] = None # Last scrape error message, if any
```

En el fichero `deal_agent_framework.py` se inicializa el rastreador:

```
def init_tracker_as_needed(self):  
    if not self.tracker:  
        from agents.product_tracker_agent import ProductTrackerAgent  
        self.log("Initializing Product Tracker Agent")  
        self.tracker = ProductTrackerAgent()  
        self.log("Product Tracker Agent ready")
```

También se añaden métodos para gestionar productos trackeados:

```

def read_tracked_products(self) -> List[TrackedProduct]:
    if os.path.exists(self.TRACKER_FILENAME):
        with open(self.TRACKER_FILENAME, "r") as f:
            data = json.load(f)
            return [TrackedProduct(**item) for item in data]
    return []

def write_tracked_products(self) -> None:
    data = [p.dict() for p in self.tracked_products]
    with open(self.TRACKER_FILENAME, "w") as f:
        json.dump(data, f, indent=2)

def add_tracked_product(
    self,
    url: str,
    target_price: Optional[float] = None,
    alert_threshold_pct: float = 10.0,
) -> TrackedProduct:
    """
    Add a new product to track. Performs an initial scrape right away
    so the UI can show the title and current price immediately.
    """
    product = TrackedProduct(
        url=url,
        target_price=target_price,
        alert_threshold_pct=alert_threshold_pct,
        added_at=datetime.now().isoformat(),
    )
    self.init_tracker_as_needed()
    product, _ = self.tracker.check_product(product)
    self.tracked_products.append(product)
    self.write_tracked_products()
    return product

def remove_tracked_product(self, url: str) -> None:
    self.tracked_products = [p for p in self.tracked_products if p.url != url]
    self.write_tracked_products()

def check_tracked_products(self) -> tuple:
    """
    Re-check prices for all tracked products.
    Returns (updated_products, alert_messages).
    """
    self.init_tracker_as_needed()

```

```

        self.tracked_products, alerts =
self.tracker.check_all(self.tracked_products)
        self.write_tracked_products()
        # Send push notifications for each alert
        if alerts and self.planner:
            for alert_text in alerts:
                try:
                    self.planner.messenger.push(alert_text)
                except Exception as e:
                    logging.warning(f"Could not send push for tracker alert: {e}")
        return self.tracked_products, alerts

def get_tracker_table(self) -> List[List]:
    """
    Return a list of rows suitable for display in a Gradio Dataframe.
    """
    rows = []
    for p in self.tracked_products:
        price_str = f"${p.current_price:.2f}" if p.current_price else "N/A"
        target_str = f"${p.target_price:.2f}" if p.target_price else "-"
        readings = len(p.price_history)
        # Simple min/max from history
        if readings >= 2:
            prices = [h["price"] for h in p.price_history]
            history_str = f"↓${min(prices):.0f} ↑${max(prices):.0f}
({readings} readings)"
        else:
            history_str = f"{readings} reading"
        last_checked = (p.last_checked or "Never")[:16].replace("T", " ")
        status = "✗" + p.scrape_error[:25] if p.scrape_error else "✓ OK"
        rows.append([
            p.title[:60],
            price_str,
            target_str,
            f"{p.alert_threshold_pct:.0f}%",
            history_str,
            last_checked,
            status,
            p.url,
        ])
    return rows

```

En el fichero price_is_righ_final.py se añade la pestaña del “Product Tracker” y la interfaz de Gradio:


```

with gr.Tab("Product Tracker"):

    # -- Add product form --
    gr.Markdown("### Track a new Amazon product")
    with gr.Row():
        url_input = gr.Textbox(
            label="Amazon Product URL",
            placeholder="https://www.amazon.com/dp/XXXXXXXXXX",
            scale=4,
        )
        target_price_input = gr.Number(
            label="Target Price ($)", value=None,
            precision=2, minimum=0, scale=1,
        )
        threshold_input = gr.Slider(
            minimum=1, maximum=50, value=10, step=1,
            label="Alert on price drop (%)", scale=1,
        )
        add_btn = gr.Button("Track", variant="primary", scale=1)

    add_status = gr.Markdown("")

    # -- Tracked products table --
    gr.Markdown("### Tracked Products")
    tracked_table = gr.Dataframe(
        headers=[
            "Product", "Current Price", "Target Price",
            "Alert %", "Price History", "Last Checked", "Status", "URL",
        ],
        wrap=True,
        column_widths=[5, 1, 1, 1, 2, 2, 1, 3],
        row_count=10, col_count=8, max_height=380,
        interactive=False,
    )

    with gr.Row():
        refresh_tracker_btn = gr.Button("Check Prices Now",
variant="primary")
        remove_btn = gr.Button("Remove Selected", variant="stop")

    tracker_alerts_html = gr.HTML()

    # -- Price history chart for selected product --

```

```

gr.Markdown("### Price History (click a row to view)")
chart_price_history = gr.Plot(show_label=False)

# State
selected_tracker_row = gr.State(None)

```

Los eventos que se manejan en Gradio son los siguientes:

Evento	Manejador	Salidas
add_btn.click	add_product()	tabla, markdown de estado
refresh_btn.click	check_prices()	tabla, HTML de alertas
tracked_table.select	on_row_select()	índice de fila seleccionada (State)
remove_btn.click	remove_product()	tabla, limpiar State de selección
ui.load	load_tracker_table()	tabla (rellenar en el primer renderizado)

Las condiciones que se evalúan para el envío de alertas son las siguientes:

1. Precio objetivo alcanzado

```

if product.target_price and current_price <= product.target_price:
    -> Enviar alerta

```

2. Caída porcentual

```

initial_price = product.price_history[0]["price"]
drop_pct = (initial_price - current_price) / initial_price * 100
if drop_pct >= product.alert_threshold_pct:

    -> Enviar alerta

```

3. Stock limitado

```

LOW_STOCK_KEYWORDS = [
    "only", "left in stock", "few left", "order soon",
    "limited availability", "1 left", "2 left", "3 left",
]
if any(kw in availability.lower() for kw in LOW_STOCK_KEYWORDS):

    -> Enviar alerta

```

Se describe a continuación el uso de la funcionalidad

Añadir un producto:

1. Abre la pestaña “Product Tracker”.
2. Pega una URL de producto de Amazon (p. ej. <https://www.amazon.es/Apple-Mini-Ordenador-Mesa-n%C3%BAcleos/dp/B0DLBRC8BJ/>).
3. Opcionalmente, establece un Precio objetivo (el precio que estás dispuesto a pagar).
4. Ajusta el deslizador de Alerta % (predeterminado: 10 — alerta si el precio baja un 10% respecto a hoy).
5. Haz clic en “Track”.

El sistema realiza un scraping inmediato y muestra el título del producto y el precio actual en la línea de estado.

Comprobar precios manualmente

Haz clic en “Check Prices Now”. Todos los productos rastreados se vuelven a extraer de forma secuencial. Las alertas aparecen en el panel rojo debajo de la tabla y también se envían como notificaciones push de Pushover (si está configurado).

Eliminar un producto

Haz clic en cualquier fila de la tabla para seleccionarla (la fila se resalta) y, a continuación, haz clic en “Eliminar” seleccionado.

3. Tools

También se crean algunos scripts a modo de herramientas para facilitar arrancar los entornos y la aplicación completa.

Están en la carpeta tools/:

1. Modal_service.sh: Gestiona el despliegue en Modal.

```
$ ./tools/modal_service.sh
```

```
Modal Service Manager — pricer-service
```

Usage:

```
./tools/modal_service.sh <command>
```

Available commands:

```
deploy → Deploy or update the service on Modal
status → List all active apps in your Modal account
logs   → Stream live logs from the app
stop   → Stop the app (frees GPU resources and avoids costs)
```

```
warm    → Keep the instance warm in a loop (every 30s)
ping    → Send a single wake_up test call
help    → Show this help message
```

2. [start.sh](#): Gestiona el arranque de la aplicación. Arranca la interfaz gráfica, el framework de agentes y el keep_warm.

```
#!/usr/bin/env bash
#
=====
=====
# start.sh — Launch the full "The Price is Right" application
#
=====
=====
#
# Usage:
# ./tools/start.sh      → Launch the full UI (Gradio + agents)
# ./tools/start.sh headless → Run the agent framework only, without UI
# ./tools/start.sh warm  → Launch UI and keep Modal warm in parallel
```